

Problem 1.1

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

Given a starting point at coordinate 0,0 - have the turtle walking in zig-zag direction starting from S and then N until a turtle pass all cells.

Output:

7,8,7,4,6,8,1,5,7,8,4,2,0,8,8,2,7,8,5,8,7,8,6,1,4,0,0,9,9,0,5,7,0,0,3,2,9,3,6,7,2,2

First, I try to hard code to find some pattern.

```
1 import numpy as np
2
3 arr_map = np.array([[7,2,0,1,0,2,9],[8,4,8,6,9,3,3],[7,8,8,8,9,0,6],[4,7,2,7,0,0,7],[6,5,7,8,0,7,2],[8,1,8,5,4,5,2]])
4 lst_pos = []
5 # print(arr_map.shape) #(6,7) ความสูง6 ความยาว7
6 h = arr_map.shape[0]
7 w = arr_map.shape[1]
8
9 for i in range(0,h,1): #ทิศลง
10     lst_pos.append([i,0])
11 for i in range(h-1,-1,-1): #ทิศขึ้น
12     lst_pos.append([i,1])
13 for i in range(0,h,1): #ทิศลง
14     lst_pos.append([i,2])
15 for i in range(h-1,-1,-1): #ทิศขึ้น
16     lst_pos.append([i,3])
17 for i in range(0,h,1): #ทิศลง
18     lst_pos.append([i,4])
19 for i in range(h-1,-1,-1): #ทิศขึ้น
20     lst_pos.append([i,5])
21 for i in range(0,h,1): #ทิศลง
22     lst_pos.append([i,6])
23 # print(lst_pos)
24
25 for pos in lst_pos:
26     print(arr_map[pos[0]][pos[1]], end=" ")
27
28 ... 7 8 7 4 6 8 1 5 7 8 4 2 0 8 8 2 7 8 5 8 7 8 6 1 4 0 0 9 9 0 5 7 0 0 3 2 2
```

I find the different of Up and Down is switched and the number of for-loop

```
1 import numpy as np
2
3 arr_map = np.array([[7,2,0,1,0,2,9],[8,4,8,6,9,3,3],[7,8,8,8,9,0,6],[4,7,2,7,0,0,7],[6,5,7,8,0,7,2],[8,1,8,5,4,5,2]])
4
5 lst_pos = []
6 h = arr_map.shape[0]
7 w = arr_map.shape[1]
8
9 for j in range(w):
10     if j%2 == 0:
11         for i in range(0,h,1): #ทิศลง
12             lst_pos.append([i,j])
13     else:
14         for i in range(h-1,-1,-1): #ทิศขึ้น
15             lst_pos.append([i,j])
16
17 for pos in lst_pos:
18     print(arr_map[pos[0]][pos[1]], end=" ")
19
20 ... [[7 2 0 1 0 2 9]
21      [8 4 8 6 9 3 3]
22      [7 8 8 8 9 0 6]
23      [4 7 2 7 0 0 7]
24      [6 5 7 8 0 7 2]
25      [8 1 8 5 4 5 2]]
26
27 7 8 7 4 6 8 1 5 7 8 4 2 0 8 8 2 7 8 5 8 7 8 6 1 4 0 0 9 9 0 5 7 0 0 3 2 2
```

All the code is reduced to shorter and can be adapted to different MxN shape if Matrix.

Problem 1.2

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

Problem 1.2

Given a starting point at coordinate X,Y - have the turtle walking to the center of the matrix in clockwise direction

Input: [1,1]

Output:

4,8,6,9,3,3,6,7,2,2,5,4,5,8,1,8,6,4,7,8,8,9,0,0,7,0,8,7,5,7,2,7,0

I create the function to find every position that's moved.

```

1 import numpy as np
2 arr_map = np.array([[7,2,0,1,0,2,9],[8,4,8,6,9,3,3],[7,8,8,8,9,0,6],[4,7,2,7,0,0,7],[6,5,7,8,0,7,2],[8,1,8,5,4,5,2]])
3 pos_start = [1,1]
4
5 def moveUp(pos,distance, lst_pos):
6     row, col = pos[0] , pos[1]
7     for i in range(1,distance+1):
8         lst_pos.append([row-i,col])
9     return(lst_pos)
10 def moveDown(pos,distance, lst_pos):
11     row, col = pos[0] , pos[1]
12     for i in range(1,distance+1):
13         lst_pos.append([row+i,col])
14     return(lst_pos)
15 def moveRight(pos,distance, lst_pos):
16     row, col = pos[0] , pos[1]
17     for i in range(1,distance+1):
18         lst_pos.append([row,col+i])
19     return(lst_pos)
20 def moveLeft(pos,distance, lst_pos):
21     row, col = pos[0] , pos[1]
22     for i in range(1,distance+1):
23         lst_pos.append([row,col-i])
24     return(lst_pos)
25

```

I try to hard code to find some pattern.

```

27
28 lst_pos = [[1,1]]
29
30 moveRight(lst_pos[-1], 5, lst_pos)
31 moveDown(lst_pos[-1], 4, lst_pos)
32 moveLeft(lst_pos[-1], 6, lst_pos)
33 moveUp(lst_pos[-1], 3, lst_pos)
34 moveRight(lst_pos[-1], 5, lst_pos)
35 moveDown(lst_pos[-1], 2, lst_pos)
36 moveLeft(lst_pos[-1], 4, lst_pos)
37 moveUp(lst_pos[-1], 1, lst_pos)
38 moveRight(lst_pos[-1], 3, lst_pos)
39
40 print(lst_pos)
41 for pos in lst_pos:
42     print(arr_map[pos[0]][pos[1]], end=" ")

```

... [[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [2, 6], [3, 6], [4, 6], [5, 6], [5, 5], [5, 4], [5, 3], [5, 2], [5, 1], [4, 1], [3, 1], [2, 1], [1, 1]]

Create another cases to find some pattern.

pos_start = [0,2]

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

pos_start = [2,0]

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

```

1 print(arr_map.shape)
2
3 lst_pos = [[0,2]]
4
5
6 moveRight(lst_pos[-1], 4, lst_pos)
7 moveDown(lst_pos[-1], 5, lst_pos)
8 moveLeft(lst_pos[-1], 6, lst_pos)
9 moveUp(lst_pos[-1], 4, lst_pos)
10 moveRight(lst_pos[-1], 5, lst_pos)
11 moveDown(lst_pos[-1], 3, lst_pos)
12 moveLeft(lst_pos[-1], 4, lst_pos)
13 moveUp(lst_pos[-1], 2, lst_pos)
14 moveRight(lst_pos[-1], 3, lst_pos)
15 moveDown(lst_pos[-1], 1, lst_pos)
16 moveLeft(lst_pos[-1], 2, lst_pos)
17
18
19 print(lst_pos)
20 for pos in lst_pos:
21     print(arr_map[pos[0]][pos[1]], end=" ")

```

(6, 7)
[[0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [1, 6], [2, 6], [3, 6], [4, 6], [5, 6],
0 1 0 2 9 3 6 7 2 2 5 4 5 8 1 8 6 4 7 8 4 8 6 9 3 0 0 7 0 8 7 5 7 8 8 8 9 0 7 2

```

1 print(arr_map.shape)
2
3 lst_pos = [[2,0]]
4
5
6 moveRight(lst_pos[-1], 6, lst_pos)
7 moveDown(lst_pos[-1], 3, lst_pos)
8 moveLeft(lst_pos[-1], 6, lst_pos)
9 moveUp(lst_pos[-1], 2, lst_pos)
10 moveRight(lst_pos[-1], 5, lst_pos)
11 moveDown(lst_pos[-1], 1, lst_pos)
12 moveLeft(lst_pos[-1], 4, lst_pos)
13
14
15 print(lst_pos)
16 for pos in lst_pos:
17     print(arr_map[pos[0]][pos[1]], end=" ")

```

(6, 7)
[[2, 0], [2, 1], [2, 2], [2, 3], [2, 4], [2, 5], [2, 6], [3, 6], [4,
7 8 8 8 9 0 6 7 2 2 5 4 5 8 1 8 6 4 7 2 7 0 0 7 0 8 7 5

But I can not find some pattern at that time...

I believe the pattern must be appear until it's found.

NO. / /
DATE / /

R C
R=5 C=6
shape = (6,7)
start = [0,0] start = [0,2] start = [2,0]

R 5 + C = 6
D 4 + r = 5
L 6 = r
U 3 + r = 4
R 5 = C - 2 = 3
D 2 + r = 3
L 4 =
U 1 + r = 2
R 3 =

R 4 + C = 6
D 5 + r = 5
L 6 = r
U 4 + r = 4
R 5 = C - 2 = 3
D 3 + r = 4
L 4 =
U 2 + r = 2
R 3 =
D 1 + r = 1
L 2 =

R 6 + C = 6
D 3 + r = 5
L 6 = r
U 2 + r = 4
R 5 = C - 2 = 3
D 1 + r = 3
L 4 =
R 5 = C = 6
C = 6

start = [1,1] start = [0,2]

R C = 5 - 1 = 4
D R + r = 6 - 1 = 5
L R
U (R-1) + r = 5 - 1 = 4
R
D
L
U
R
D
L
U

C - C = 6 - 1 = 5
R - r = 5 - 1 = 4
R + 1 = 6
(R-1) - r = 4 - 1 = 3
C - 1 = 5
2
4
→ 1
→ 3 stop

C - C = 6 - 2 = 4
R - r = 5 - 0 = 5
R + 1 = 5 + 1 = 6
(R-1) - r = 4 - 0 = 4
C - 1 = 5
3
4
2
3
1
2

The all moving functions are reduced again after the pattern was found.

```
1 import numpy as np
2
3 def Move(numMove, pos, distance, lst_pos):
4     row, col = pos[0] , pos[1]
5     for i in range(1,distance+1):
6         if numMove == 0:
7             lst_pos.append([row,col+i])
8         elif numMove == 1:
9             lst_pos.append([row+i,col])
10        elif numMove == 2:
11            lst_pos.append([row,col-i])
12        elif numMove == 3:
13            lst_pos.append([row-i,col])
14    return(lst_pos)
15
16
```

The pattern was apple to code.

```
18 arr_map = np.array([[7,2,0,1,0,2,9],[8,4,8,6,9,3,3],[7,8,8,8,9,0,6],[4,7,2,7,0,
19 print("Enter the point in form num1,num2.")
20 print("[1,2] must be 1,2 ")
21 pos_txt = input("Enter the point in form num1,num2 : ").split(",")
22 pos_start = [int(pos_txt[0]),int(pos_txt[1])]
23 print(pos_start)
24
25
26
27 R, C = arr_map.shape[0]-1, arr_map.shape[1]-1
28 lst_pos = [pos_start]
29 r, c = pos_start[0], pos_start[1]
30 #create list distance
31 lst_distance = [C-c,R-r,R+1,(R-1)-r,C-1]
32 i = 1
33 while True:
34     lst_distance.append(lst_distance[i]-2)
35     if lst_distance[-2]==1:
36         break
37     i += 1
38 # print(lst_distance)
39
40
41 #create list of point
42 for i in range(len(lst_distance)):
43     Move(i%4, lst_pos[-1], lst_distance[i], lst_pos)
44
45 # print(lst_pos)
46 for pos in lst_pos:
47     print(arr_map[pos[0]][pos[1]], end=" ")
```

Problem 1.2

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

Problem 1.3

Given that a turtle can travel in direction of only N E S W in each trip and the starting value - have the turtle find the target number without changing direction using the shortest and longest route. If the route can't be found, please display "NO ROUTE"

Input: [2,8]

Output:

N 2,8 SHORTEST

S 2,4,8

S 2,7,8

W 2,5,4,5,8,1,8 LONGEST

The first plan is to create 4 directions of moving to check every point that is origin such as 2.

```

1 import numpy as np
2
3 arr_map = np.array([[7,2,0,1,0,2,9],[8,4,8,6,9,3,3],[7,8,8,8,9,0,6],
4                     [4,7,2,7,0,0,7],[6,5,7,8,0,7,2],[8,1,8,5,4,5,2]])
5 h , w = arr_map.shape[0]-1, arr_map.shape[1]-1
6
7 pos_now = [5,6]
8
9 lst_pos = []
10 ##E
11 # for i in range(pos_now[1],w,1):
12 #     lst_pos.append([pos_now[0], i+1])
13 #W
14 for i in range(pos_now[1]-1,-1,-1):
15     lst_pos.append([pos_now[0], i])
16 ##N
17 # for i in range(pos_now[0]-1,-1,-1):
18 #     lst_pos.append([i, pos_now[1]])
19 ##S
20 # for i in range(pos_now[0],h,1):
21 #     lst_pos.append([ i+1, pos_now[1]])
22

```

7	2	0	1	0	2	9
8	4	8	6	9	3	3
7	8	8	8	9	0	6
4	7	2	7	0	0	7
6	5	7	8	0	7	2
8	1	8	5	4	5	2

Problem 1.3

Given that a turtle can travel in a straight line without changing direction, the route can't be found, p

Input: [2,8]

Output:

N 2,8 SHORTEST

S 2,4,8

S 2,7,8

W 2,5,4,5,8,1,8 LONGEST

Now I find more sequence: W 2, 7, 0, 8.